

Subgraphs in LangGraph

Modularity • Multi-Agent • State Isolation • Composition

Detailed Notes with Code & Extra Content from LangGraph Docs

CampusX Lecture Series

Subgraph | Parent Graph | State Isolation | Multi-Agent | Shared State

■ TABLE OF CONTENTS ■

01	What Are Subgraphs?	*Definition & visual concept*
02	The Need for Subgraphs	*Complexity problem & analogies*
03	Benefits of Subgraphs	*Modularity, reuse, isolation*
04	Method 1: Invoke from Node	*Isolated states, full code*
05	Method 2: Graph as Node	*Shared state, direct mounting*
06	Method Comparison	*When to use which approach*
07	Advanced: State Transformation	*Mapping between schemas (from Docs)*
08	Advanced: Multi-Agent Systems	*Subgraphs for agent teams (from Docs)*
09	Advanced: Persistence & Streaming	*Checkpointers in subgraphs (from Docs)*
10	Advanced: Map-Reduce Pattern	*Dynamic parallel tasks (from Docs)*
11	Anti-Patterns & Best Practices	*Common mistakes to avoid*
12	Summary & Quick Reference	*Everything at a glance*

01 What Are Subgraphs?

A **Subgraph** in LangGraph is a graph that is embedded and executed as a **node** inside another graph (the "Parent Graph"). It contains its own nodes, edges, logic, and optionally its own state — but from the parent's perspective, it looks like a single node.

■ Without Subgraphs	✔ With Subgraphs
• One massive graph with dozens of nodes • All nodes share a single global State • Hard to debug — one failure breaks everything • Hard to maintain — changes ripple everywhere • **Chaos at scale**	• Parent graph coordinates high-level flow • Each subgraph has its own internal logic • Failures are isolated to the subgraph • Clean state separation between components • **Scalable & maintainable**

■ Foundation for Multi-Agent AI

This is the **foundational concept** required for building **Multi-Agent Systems**. Each agent becomes a subgraph with its own tools, memory, and reasoning logic, while a parent graph orchestrates the coordination between agents.

02 The Need for Subgraphs

The Complexity Problem:

Real-world AI applications start simple (Query → LLM → Response) but quickly accumulate complexity: tools, RAG, conditional routing, retry logic, memory, HITL, guardrails, and evaluation. A single flat graph becomes unmanageable.

■ The "Software Developer Agent" Analogy

Imagine building an autonomous **Software Development Agent**. This process involves multiple distinct roles, each needing different tools, logic, and state:

- **Team Lead:** Plans architecture, assigns tasks, reviews progress
- **Backend Team:** Writes server code, needs DB tools, has retry logic
- **Frontend Team:** Writes UI code, needs component libraries
- **Tester:** Runs tests, needs test frameworks, reports bugs
- **DevOps:** Deploys code, needs CI/CD tools, different guardrails

If you build this as **one single graph**, it becomes chaos. The Tester sees the Backend's scratchpad. The DevOps node shares state with the Frontend. Debugging is a nightmare.

Solution: Each team becomes a **Subgraph** with its own internal logic, tools, and memory. The Parent Graph (Team Lead) coordinates between them.

03 Benefits of Subgraphs

A. Conceptual Benefits (Software Engineering Principles):

■ Modularity

Break complex workflows into manageable, self-contained units. Just as you split code into functions/classes, you split graphs into subgraphs.

■ Reusability

Build a "Coding Subgraph" once, reuse it for Backend AND Frontend teams — just feed different inputs. Build once, deploy many times.

■ Maintainability

If testing fails, debug ONLY the Testing Subgraph. No need to trace through 50 nodes in a monolithic graph.

B. LangGraph-Specific Benefits:

■ Failure Isolation

If a subgraph crashes, the parent graph can handle the error gracefully (try/except around invoke). In a monolithic graph, one node failure can cascade and crash everything.

■ State Separation (CRUCIAL)

Parent has its state; subgraph has its **own independent state**. They only exchange necessary data (inputs/outputs). The Tester agent never sees the Backend's temporary scratchpad variables. Clean boundaries.

■ Observability (Tracing)

LangSmith traces subgraphs independently. Analyze exactly how many tokens the "Coding Agent" used vs the "DevOps Agent". Granular cost tracking and performance monitoring per sub-agent.

04 Method 1: Invoke from Node

■ Isolated States — Full Independence

In this method, the parent and subgraph are **completely independent**. The parent has a normal node function that **manually invokes** the subgraph inside it. States are separate — you must explicitly transform data between them.

STEP 1: DEFINE THE SUBGRAPH (Translator)

```
from langgraph.graph import StateGraph, START, END
from typing import TypedDict

# 1. Subgraph has its OWN state (independent!)
class SubState(TypedDict):
    input_text: str       # Input from Parent
    translated_text: str  # Output to Parent

# 2. Subgraph logic
def translate_text(state: SubState):
    english = state["input_text"]
    result = f"Hindi translation of: {english}"
    return {"translated_text": result}

# 3. Build & compile subgraph
sub_builder = StateGraph(SubState)
sub_builder.add_node("translate_node", translate_text)
sub_builder.add_edge(START, "translate_node")
sub_builder.add_edge("translate_node", END)
subgraph = sub_builder.compile()
```

STEP 2: DEFINE PARENT GRAPH (with wrapper node)

```
# Parent has DIFFERENT state
class ParentState(TypedDict):
    question: str
    english_answer: str
    hindi_answer: str

def generate_answer(state: ParentState):
    return {"english_answer": "Answer in English."}

# ■ THE WRAPPER: transforms state & invokes subgraph
def call_subgraph_node(state: ParentState):
    english_input = state["english_answer"]

    # Transform Parent State → Subgraph State
    subgraph_output = subgraph.invoke(
        {"input_text": english_input}
    )

    # Transform Subgraph Output → Parent State
    return {"hindi_answer": subgraph_output["translated_text"]}

# Build parent
builder = StateGraph(ParentState)
builder.add_node("generate", generate_answer)
builder.add_node("translate_wrapper", call_subgraph_node)
builder.add_edge(START, "generate")
```

```
builder.add_edge("generate", "translate_wrapper")
builder.add_edge("translate_wrapper", END)
parent_graph = builder.compile()
```

05 Method 2: Graph as Node

■ Shared State — Direct Mounting

In this method, you add the **compiled subgraph directly** as a node using `add_node()`. The subgraph usually **shares state keys** with the parent. No wrapper function needed — state flows automatically.

SHARED STATE + SUBGRAPH + PARENT (Complete)

```
# Single shared state definition
class SharedState(TypedDict):
    question: str
    english_answer: str
    hindi_answer: str

# Subgraph logic (directly uses shared keys)
def translate_logic(state: SharedState):
    text = state["english_answer"]
    return {"hindi_answer": f"Hindi: {text}"}

# Build subgraph
sub_builder = StateGraph(SharedState)
sub_builder.add_node("translate_worker", translate_logic)
sub_builder.add_edge(START, "translate_worker")
sub_builder.add_edge("translate_worker", END)
subgraph_shared = sub_builder.compile()

# Parent logic
def generate_logic(state: SharedState):
    return {"english_answer": "English answer."}

# Build parent — ■ pass compiled subgraph directly!
builder = StateGraph(SharedState)
builder.add_node("generate", generate_logic)
builder.add_node("translate_subgraph", subgraph_shared) # ■
builder.add_edge(START, "generate")
builder.add_edge("generate", "translate_subgraph")
builder.add_edge("translate_subgraph", END)
app = builder.compile()
```

■ The Magic Line

Key Difference: Instead of a function, you pass the **compiled subgraph object** to `add_node()`. LangGraph automatically routes the shared state keys through the subgraph. No manual transformation needed!

06 Method Comparison

Feature	Method 1: Invoke from Node	Method 2: Graph as Node
State	Completely separate	Shared keys
Syntax	`subgraph.invoke({...})`	`add_node("x", subgraph)`
Transformation	Manual (you map in/out)	Automatic (via shared keys)
Isolation	✔ Full isolation	■ Shared state
Complexity	More code (wrapper needed)	Less code (simpler)
Best For	**Distinct agents, different teams**	**Consistent data pipelines**
Private State	✔ Fully private	Only non-shared keys are private

■ Which Method to Choose?

Decision Rule: If the subgraph needs its own independent schema (e.g., a multi-agent system where each agent tracks private data), use **Method 1**. If the data structure is consistent throughout the pipeline (like a translation pipeline), use **Method 2** for simplicity.

07 Adv: State Transformation

EXTRA CONTENT — From LangGraph Documentation

When the parent and subgraph have **completely different schemas** (no shared keys), you must use a wrapper node that transforms state in both directions. This is the standard pattern from the official LangGraph documentation:

STATE TRANSFORMATION PATTERN (from LangGraph Docs)

```
# Subgraph with DIFFERENT schema (no overlap with parent)
class SubgraphState(TypedDict):
    bar: str  # No overlap with parent's 'foo'

def subgraph_node_1(state: SubgraphState):
    return {"bar": "hi! " + state["bar"]}

subgraph_builder = StateGraph(SubgraphState)
subgraph_builder.add_node(subgraph_node_1)
subgraph_builder.add_edge(START, "subgraph_node_1")
subgraph = subgraph_builder.compile()

# Parent with DIFFERENT schema
class State(TypedDict):
    foo: str

def call_subgraph(state: State):
    # ■ Transform: Parent → Subgraph
    subgraph_output = subgraph.invoke({"bar": state["foo"]})
    # ■ Transform: Subgraph → Parent
    return {"foo": subgraph_output["bar"]}

builder = StateGraph(State)
builder.add_node("node_1", call_subgraph)
builder.add_edge(START, "node_1")
graph = builder.compile()
```

■ Private + Shared State (from Docs)

Hybrid Pattern (Private + Shared Keys): A subgraph can have **both** shared keys (overlapping with parent) and private keys (only visible inside the subgraph). When mounted directly via `add_node()`, only shared keys flow to/from the parent. Private keys exist only during subgraph execution.

```
# Subgraph: 'foo' is shared, 'bar' is private
class SubgraphState(TypedDict):
    foo: str  # shared with parent
    bar: str  # private to subgraph
```

08 Adv: Multi-Agent Systems

EXTRA CONTENT — From LangGraph Documentation

The primary production use case for subgraphs is **multi-agent systems**. Each agent is a subgraph with its own tools, memory, and reasoning — coordinated by a parent graph.

Multi-Agent Architecture Patterns:

■ Supervisor Pattern

A "supervisor" agent (parent graph) decides which specialist agent (subgraph) to call next based on the task. The supervisor routes work and aggregates results.

■ *Supervisor → Research Agent / Coding Agent / Writing Agent*

■ Handoff Pattern

Agents can transfer control to each other using `Command(goto="bob", graph=Command.PARENT)`. An agent in one subgraph can directly route to a different subgraph via the parent.

■ *Alice finishes → transfers to Bob → Bob completes*

■ Hierarchical Teams

Subgraphs can contain their OWN subgraphs (nesting!). A "Research Team" subgraph might contain a "Search Agent" and a "Summary Agent" as sub-subgraphs.

■ *Parent → Research Team → [Search Agent, Summary Agent]*

HANDOFF BETWEEN SUBGRAPHS (from Docs)

```
# Inside a node of subgraph 'alice':
from langgraph.types import Command

def some_node_inside_alice(state):
    return Command(
        goto="bob",                # Navigate to 'bob' subgraph
        update={"messages": ["handoff data"]},
        graph=Command.PARENT       # Navigate in PARENT graph
    )
```

09 Adv: Persistence & Streaming

EXTRA CONTENT — From LangGraph Documentation

■ Persistence: Checkpointers in Subgraphs

When you add a checkpointer to the **parent graph**, LangGraph automatically handles persistence for all subgraphs too. You generally do **NOT** need separate checkpointers for child subgraphs — the parent's checkpointer covers everything.

However, if you want a subgraph to have its **own independent memory** (e.g., each agent tracking its own private message history across sessions), you CAN compile the subgraph with its own checkpointer.

PARENT CHECKPOINTER COVERS SUBGRAPHS

```
from langgraph.checkpoint.memory import MemorySaver

# Subgraph compiled WITHOUT its own checkpointer
subgraph = sub_builder.compile()

# Parent compiled WITH checkpointer — covers everything
checkpointer = MemorySaver()
graph = builder.compile(checkpointer=checkpointer)

# For independent subgraph memory (advanced):
subgraph = sub_builder.compile(
    checkpointer=MemorySaver() # Own memory!
)
```

■ Streaming: Observing Subgraph Events

You can stream events from **inside subgraphs** up to the parent by passing `subgraphs=True` to the stream method. Events are namespaced with an invocation ID so you know which subgraph generated them.

STREAMING SUBGRAPH EVENTS

```
# Pass subgraphs=True to see events from inside subgraphs
for chunk in graph.stream({"foo": "input"}, subgraphs=True):
    print(chunk)

# Output shows namespace for subgraph events:
# ((), {'node_1': {'foo': 'hi! foo'}})          ← parent
# (('node_2:abc123',), {'sub_node': {'bar': ...}}) ← subgraph
# ((), {'node_2': {'foo': 'final result'}})      ← parent
```

■ State Inspection

Inspecting Subgraph State: You can access the specific state of a subgraph's history using the `get_state()` method on the parent graph. LangSmith also shows subgraph traces with token usage breakdowns per sub-agent — critical for cost tracking.

10 Adv: Map-Reduce Pattern

EXTRA CONTENT — From LangGraph Documentation

While subgraphs handle fixed, design-time composition, LangGraph also supports **dynamic task creation at runtime** using the **Send API**. This implements the Map-Reduce pattern where the number of parallel tasks is determined by the graph's state.

■ Map-Reduce with Send API

Map Phase: Split work into independent units that process **in parallel**. Each unit can be a subgraph invocation with different inputs.

Reduce Phase: Combine all parallel outputs into a single aggregated result using a reducer function (like `add_messages`).

Example: A research agent that dynamically searches 5 different sources in parallel, then a summarizer agent combines all results into one report.

DYNAMIC PARALLEL EXECUTION (Send API)

```python
from langgraph.types import Send

def router_node(state):
    # Dynamically create parallel tasks based on state
    topics = state["topics_to_research"]

    # Send spawns a parallel task for each topic
    return [
        Send("research_node", {"topic": topic})
        for topic in topics
    ]

# Each Send creates a parallel execution of 'research_node'
# Results are aggregated using the state's reducer function
```

Subgraphs vs Map-Reduce:

Aspect	Subgraphs	Map-Reduce (Send)
Task count	Fixed at design time	**Dynamic at runtime**
Execution	Sequential (by default)	**Parallel**
Use case	Multi-agent teams	Fan-out research/analysis

11 Anti-Patterns & Best Practices

■ Over-Subgraphing

Don't create a subgraph for every single node. Subgraphs add overhead (state serialization, invocation cost). Only use them when you genuinely need isolation, reusability, or modularity.

Fix: If a task is 1-2 nodes with no independent state needs, keep it as regular nodes.

■ Leaking Internal State

With shared-state subgraphs (Method 2), the subgraph can see AND modify ALL parent state keys. A subgraph accidentally overwriting a parent key can cause subtle, hard-to-debug errors.

Fix: Use Method 1 (invoke) for strict isolation, or keep subgraph private keys separate.

■ No Error Handling on Invoke

When invoking a subgraph from a wrapper node, a crash inside the subgraph will propagate up and crash the parent. The whole point of isolation is lost.

Fix: Wrap `subgraph.invoke()` in try/except inside the wrapper node.

■ Multiple Subgraph Invocations in One Node

You **cannot invoke more than one subgraph** inside the same node if checkpointing is enabled. This is a documented LangGraph limitation.

Fix: Split into separate nodes, each invoking one subgraph.

■ Ignoring Observability

Running subgraphs without tracing makes debugging multi-agent systems nearly impossible. You can't tell which sub-agent consumed the most tokens or where failures occurred.

Fix: Use LangSmith tracing + `subgraphs=True` in stream calls.

12 Summary & Quick Reference

Concept	Purpose	Code
Subgraph	Graph embedded as a node in parent	`sub_builder.compile()`
Method 1 (Invoke)	Isolated states, manual transform	`subgraph.invoke({...})`
Method 2 (Direct)	Shared state, auto-routing	`add_node("x", subgraph)`
Handoff	Transfer between agents	`Command(goto=, graph=PARENT)`
Streaming	Observe inside subgraphs	`stream(..., subgraphs=True)`
Map-Reduce	Dynamic parallel tasks	`Send("node", {...})`

Supervisor	Handoff	Hierarchical
Central coordinator routes to agents	Agents transfer to each other	Nested subgraphs teams within teams

The Big Takeaway

Subgraphs are the **building blocks of scalable AI systems**. They transform a monolithic graph into a modular architecture where each component has clear boundaries, independent state, and focused responsibility. Whether you use isolated invocation (Method 1) or direct mounting (Method 2), the result is the same: **clean, maintainable, production-ready multi-agent AI**.

One graph to rule them all — composed of many focused subgraphs.

Parent Graph (Coordinator) + **Subgraphs** (Specialist Agents) = **Multi-Agent System!**